

5. ArrayList与顺序表

【本节目标】

1. 线性表
2. 顺序表
3. ArrayList的简介
4. ArrayList使用
5. ArrayList的扩容机制
6. 扑克牌

1. 线性表

线性表 (*linear list*) 是n个具有相同特性的数据元素的有限序列。线性表是一种在实际中广泛使用的数据结构，常见的线性表：顺序表、链表、栈、队列...

线性表在逻辑上是线性结构，也就是说是一条连续的一条直线。但是在物理结构上并不一定是连续的，线性表在物理上存储时，通常以数组和链式结构的形式存储。



顺序表



链表

2. 顺序表

顺序表是用一段**物理地址连续**的存储单元依次存储数据元素的线性结构，一般情况下采用数组存储。在数组上完成数据的增删查改。

2.1 接口的实现

```
1 public class SeqList {
```

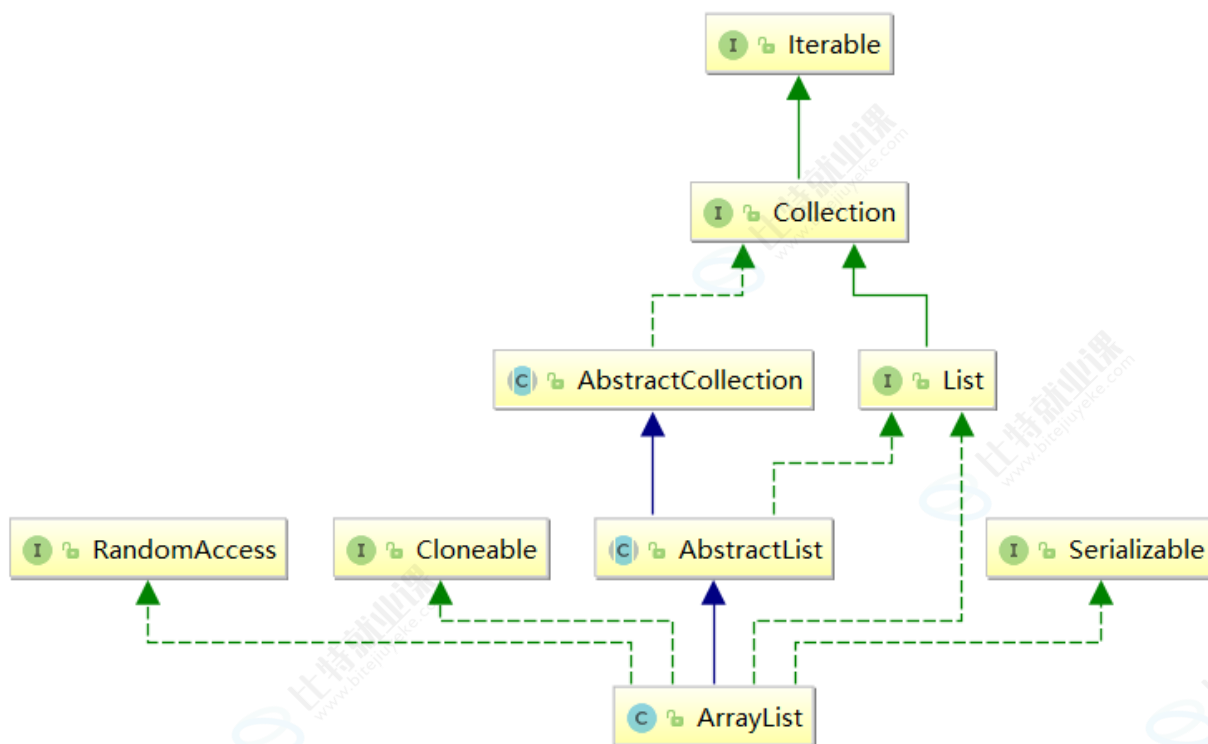
```

2     private int[] array;
3     private int size;
4     // 默认构造方法
5     SeqList(){    }
6     // 将顺序表的底层容量设置为initcapacity
7     SeqList(int initcapacity){    }
8
9     // 新增元素,默认在数组最后新增
10    public void add(int data) {    }
11    // 在 pos 位置新增元素
12    public void add(int pos, int data) {    }
13    // 判定是否包含某个元素
14    public boolean contains(int toFind) { return true; }
15    // 查找某个元素对应的位置
16    public int indexOf(int toFind) { return -1; }
17    // 获取 pos 位置的元素
18    public int get(int pos) { return -1; }
19    // 给 pos 位置的元素设为 value
20    public void set(int pos, int value) {    }
21    //删除第一次出现的关键字key
22    public void remove(int toRemove) {    }
23    // 获取顺序表长度
24    public int size() { return 0; }
25    // 清空顺序表
26    public void clear() {    }
27
28    // 打印顺序表, 注意: 该方法并不是顺序表中的方法, 为了方便看测试结果给出的
29    public void display() {    }
30 }

```

3. ArrayList简介

在集合框架中, ArrayList是一个普通的类, 实现了List接口, 具体框架图如下:



【说明】

1. ArrayList是以泛型方式实现的，使用时必须要先实例化
2. ArrayList实现了RandomAccess接口，表明ArrayList支持随机访问
3. ArrayList实现了Cloneable接口，表明ArrayList是可以clone的
4. ArrayList实现了Serializable接口，表明ArrayList是支持序列化的
5. 和Vector不同，ArrayList不是线程安全的，在单线程下可以使用，在多线程中可以选择Vector或者CopyOnWriteArrayList
6. ArrayList底层是一段连续的空间，并且可以动态扩容，是一个动态类型的顺序表

4. ArrayList使用

4.1 ArrayList的构造

方法	解释
ArrayList()	无参构造
ArrayList(Collection<? extends E> c)	利用其他 Collection 构建 ArrayList
ArrayList(int initialCapacity)	指定顺序表初始容量

```
1 public static void main(String[] args) {
2     // ArrayList创建，推荐写法
3     // 构造一个空的列表
4     List<Integer> list1 = new ArrayList<>();
```

```

5
6    // 构造一个具有10个容量的列表
7    List<Integer> list2 = new ArrayList<>(10);
8    list2.add(1);
9    list2.add(2);
10   list2.add(3);
11   // list2.add("hello"); // 编译失败, List<Integer>已经限定了, list2中只能存储整
    形元素
12
13   // list3构造好之后, 与list中的元素一致
14   ArrayList<Integer> list3 = new ArrayList<>(list2);
15
16   // 避免省略类型, 否则: 任意类型的元素都可以存放, 使用时将是一场灾难
17   List list4 = new ArrayList();
18   list4.add("111");
19   list4.add(100);
20 }

```

4.2 ArrayList常见操作

ArrayList虽然提供的方法比较多, 但是常用方法如下所示, 需要用到其他方法时, 同学们自行查看ArrayList的帮助文档。

方法	解释
boolean add (E e)	尾插 e
void add (int index, E element)	将 e 插入到 index 位置
boolean addAll (Collection<? extends E> c)	尾插 c 中的元素
E remove (int index)	删除 index 位置元素
boolean remove (Object o)	删除遇到的第一个 o
E get (int index)	获取下标 index 位置元素
E set (int index, E element)	将下标 index 位置元素设置为 element
void clear ()	清空
boolean contains (Object o)	判断 o 是否在线性表中
int indexOf (Object o)	返回第一个 o 所在下标
int lastIndexOf (Object o)	返回最后一个 o 的下标
List<E> subList (int fromIndex, int toIndex)	截取部分 list

```
1 public static void main(String[] args) {
2     List<String> list = new ArrayList<>();
3     list.add("JavaSE");
4     list.add("JavaWeb");
5     list.add("JavaEE");
6     list.add("JVM");
7     list.add("测试课程");
8     System.out.println(list);
9
10    // 获取list中有效元素个数
11    System.out.println(list.size());
12
13    // 获取和设置index位置上的元素, 注意index必须介于[0, size)间
14    System.out.println(list.get(1));
15    list.set(1, "JavaWEB");
16    System.out.println(list.get(1));
17
18    // 在list的index位置插入指定元素, index及后续的元素统一往后搬移一个位置
19    list.add(1, "Java数据结构");
20    System.out.println(list);
21    // 删除指定元素, 找到了就删除, 该元素之后的元素统一往前搬移一个位置
22    list.remove("JVM");
23    System.out.println(list);
24    // 删除list中index位置上的元素, 注意index不要超过list中有效元素个数, 否则会抛出下标
    越界异常
25    list.remove(list.size()-1);
26    System.out.println(list);
27
28    // 检测list中是否包含指定元素, 包含返回true, 否则返回false
29    if(list.contains("测试课程")){
30        list.add("测试课程");
31    }
32
33    // 查找指定元素第一次出现的位置: indexOf从前往后找, lastIndexOf从后往前找
34    list.add("JavaSE");
35    System.out.println(list.indexOf("JavaSE"));
36    System.out.println(list.lastIndexOf("JavaSE"));
37
38    // 使用list中[0, 4)之间的元素构成一个新的SubList返回, 但是和ArrayList共用一个
    elementData数组
39    List<String> ret = list.subList(0, 4);
40    System.out.println(ret);
41
42    list.clear();
43    System.out.println(list.size());
44 }
```

4.3 ArrayList的遍历

ArrayList 可以使用三方方式遍历：for循环+下标、foreach、使用迭代器

```
1 public static void main(String[] args) {
2     List<Integer> list = new ArrayList<>();
3     list.add(1);
4     list.add(2);
5     list.add(3);
6     list.add(4);
7     list.add(5);
8     // 使用下标+for遍历
9     for (int i = 0; i < list.size(); i++) {
10         System.out.print(list.get(i) + " ");
11     }
12     System.out.println();
13     // 借助foreach遍历
14     for (Integer integer : list) {
15         System.out.print(integer + " ");
16     }
17     System.out.println();
18     Iterator<Integer> it = list.listIterator();
19     while(it.hasNext()){
20         System.out.print(it.next() + " ");
21     }
22     System.out.println();
23 }
```

注意：

1. ArrayList最常使用的遍历方式是：for循环+下标 以及 foreach
2. 迭代器是设计模式的一种，后序容器接触多了再给大家铺垫

4.4 ArrayList的扩容机制

下面代码有缺陷吗？为什么？

```
1 public static void main(String[] args) {
2     List<Integer> list = new ArrayList<>();
3     for (int i = 0; i < 100; i++) {
4         list.add(i);
5     }
6 }
```

ArrayList是一个动态类型的顺序表，即：在插入元素的过程中会自动扩容。以下是ArrayList源码(JDK17)中扩容方式：

```
1 public boolean add(E e) {
2     modCount++;
3     add(e, elementData, size);
4     return true;
5 }
6
7 private void add(E e, Object[] elementData, int s) {
8     if (s == elementData.length) //第一次存储的时候 elementData.length=0, s = 0
9         elementData = grow();
10    elementData[s] = e;
11    size = s + 1;
12 }
13
14 private Object[] grow() {
15     return grow(size + 1);
16 }
17
18 private Object[] grow(int minCapacity) {
19     int oldCapacity = elementData.length;
20     if (oldCapacity > 0 || elementData != DEFAULTCAPACITY_EMPTY_ELEMENTDATA) {
21         int newCapacity = ArraysSupport.newLength(oldCapacity,
22             minCapacity - oldCapacity, /* minimum growth */
23             oldCapacity >> 1 /* preferred growth */);
24         return elementData = Arrays.copyOf(elementData, newCapacity);
25     } else {
26         return elementData = new Object[Math.max(DEFAULT_CAPACITY,
27             minCapacity)];
28     }
29
30
31 //oldLength: 当前数组的长度
32 //minGrowth: 最小需要增加的长度
33 //prefGrowth: 首选的增长长度 (通常是当前大小的一半)
34 public static int newLength(int oldLength, int minGrowth, int prefGrowth) {
35     // preconditions not checked because of inlining
36     // assert oldLength >= 0
37     // assert minGrowth > 0
38
39     //当前长度加上 minGrowth 和 prefGrowth 中的较大值。
40     int prefLength = oldLength + Math.max(minGrowth, prefGrowth); // might
41     overflow
42     if (0 < prefLength && prefLength <= SOFT_MAX_ARRAY_LENGTH) {
```

```

42         return prefLength;
43     } else {
44         // put code cold in a separate method
45         return hugeLength(oldLength, minGrowth);
46     }
47
48
49
50 }
51
52 private static int hugeLength(int oldLength, int minGrowth) {
53     int minLength = oldLength + minGrowth;
54     if (minLength < 0) { // overflow
55         throw new OutOfMemoryError(
56             "Required array length " + oldLength + " + " + minGrowth + " is
too large");
57     } else if (minLength <= SOFT_MAX_ARRAY_LENGTH) {
58         return SOFT_MAX_ARRAY_LENGTH;
59     } else {
60         return minLength;
61     }
62 }

```

- 如果调用不带参数的构造方法，第一次分配数组大小为10
- 后续扩容为1.5倍扩容

5. ArrayList的具体使用

5.1 简单的洗牌算法

```

1 public class Card {
2     public int rank;        // 牌面值
3     public String suit;     // 花色
4
5     @Override
6     public String toString() {
7         return String.format("[%s %d]", suit, rank);
8     }
9 }

```

```

1 import java.util.List;
2 import java.util.ArrayList;

```



```

3 import java.util.Random;
4
5 public class CardDemo {
6     public static final String[] SUITS = {"♠", "♥", "♣", "♦"};
7     // 买一副牌
8     private static List<Card> buyDeck() {
9         List<Card> deck = new ArrayList<>(52);
10        for (int i = 0; i < 4; i++) {
11            for (int j = 1; j <= 13; j++) {
12                String suit = SUITS[i];
13                int rank = j;
14                Card card = new Card();
15                card.rank = rank;
16                card.suit = suit;
17
18                deck.add(card);
19            }
20        }
21
22        return deck;
23    }
24
25    private static void swap(List<Card> deck, int i, int j) {
26        Card t = deck.get(i);
27        deck.set(i, deck.get(j));
28        deck.set(j, t);
29    }
30
31    private static void shuffle(List<Card> deck) {
32        Random random = new Random(20190905);
33        for (int i = deck.size() - 1; i > 0; i--) {
34            int r = random.nextInt(i);
35            swap(deck, i, r);
36        }
37    }
38
39    public static void main(String[] args) {
40        List<Card> deck = buyDeck();
41        System.out.println("刚买回来的牌:");
42        System.out.println(deck);
43        shuffle(deck);
44        System.out.println("洗过的牌:");
45        System.out.println(deck);
46        // 三个人, 每个人轮流抓 5 张牌
47        List<List<Card>> hands = new ArrayList<>();
48        hands.add(new ArrayList<>());
49        hands.add(new ArrayList<>());

```

```
50     hands.add(new ArrayList<>());
51
52     for (int i = 0; i < 5; i++) {
53         for (int j = 0; j < 3; j++) {
54             hands.get(j).add(deck.remove(0));
55         }
56     }
57
58     System.out.println("剩余的牌:");
59     System.out.println(deck);
60     System.out.println("A 手中的牌:");
61     System.out.println(hands.get(0));
62     System.out.println("B 手中的牌:");
63     System.out.println(hands.get(1));
64     System.out.println("C 手中的牌:");
65     System.out.println(hands.get(2));
66 }
67 }
```

5.2 杨辉三角

118. 杨辉三角 - 力扣 (LeetCode)

6. ArrayList的问题及思考

1. ArrayList底层使用连续的空间，任意位置插入或删除元素时，需要将该位置后序元素整体往前或者往后搬移，故时间复杂度为 $O(N)$
2. 增容需要申请新空间，拷贝数据，释放旧空间。会有不小的消耗。
3. 增容一般是呈2倍的增长，势必会有一定的空间浪费。例如当前容量为100，满了以后增容到200，我们再继续插入了5个数据，后面没有数据插入了，那么就浪费了95个数据空间。

思考：如何解决以上问题呢？

完